Architecture Decision Records

Agentic PR Review & Release Notes Pipeline — ADR Package

Contents

Each record follows the same structure: context, decision, rejected alternatives (with evidence where applicable), evidence, consequences, and open risks — per this project's ADR quality bar.

ADR-001: Convert Risk-Scoring from Agent Skill to Deterministic Code

Status: *Accepted*

Date: *2026-08-05*

Context

`skills/risk-scoring.md` defines the mapping from a reviewer agent's `findings` (severity per issue) to an `overall_risk_score` and `overall_recommendation`. As originally designed, this was a "skill" the reviewer agent would apply via its own reasoning on every run.

The baseline review (`docs/baseline-metrics.md`) found that manual human review scored 1/2 on "Risk Flagging" across all 3 seeded-bug PRs — humans correctly identified *what* was wrong but were inconsistent about stating severity and recommended action. `skills/risk-scoring.md` was written specifically to fix this by giving explicit rules. But leaving those rules to be *applied* by an LLM agent's reasoning, rather than run as code, risks reintroducing the same inconsistency the rules were meant to eliminate — an agent could still reason its way to a slightly different conclusion on a similar case, run to run.

Examining the actual rules in `skills/risk-scoring.md` revealed they are a complete, unambiguous lookup table (any critical -> escalate; any high -> escalate; only medium/low -> request_changes if medium present else approve; none -> approve). There is no genuine judgment call left once `findings` exist — the mapping itself is arithmetic (find the max severity, look up the corresponding action).

Decision

Convert the risk-scoring mapping from an agent-invoked skill into deterministic Python code (`skills/risk_scoring.py`). The reviewer agent still performs the actual judgment-heavy work (reading the diff, deciding what's wrong, assigning a severity to each finding) — only the final mechanical mapping step moves to code.

Rejected Alternatives

Alternative 1: Keep it as an agent-applied skill (status quo). Rejected because it doesn't structurally prevent the exact inconsistency problem it was built to solve — evidence: `docs/baseline-metrics.md` shows this is a real, measured weakness in this project's specific domain (Risk Flagging scored 1/2 across all 3 seeded-bug baseline reviews), and there was no mechanism to guarantee an LLM reliably applies a fixed lookup table identically every time.

Alternative 2: Use an ML classifier instead of a hand-written rule table. Rejected as unnecessary complexity — the rule table is small, fully specified, and has no ambiguous cases requiring statistical inference. Evidence: all 9 branch combinations in `skills/risk_scoring.py`'s self-test are exhaustively covered by 4 simple conditional branches; there's no pattern here that benefits from a learned model over an explicit lookup.

Alternative 3: Keep it agent-applied, but add a stricter prompt/example set to improve consistency. Considered as a middle ground, but rejected because even a well-prompted LLM call remains non-deterministic in principle (temperature, model version drift) and still costs real money and latency per call for zero added judgment value — evidence: the before/after comparison in `docs/before-after-risk-scoring-conversion.md` shows the deterministic version runs in ~0.001ms at $0 marginal cost versus a full LLM API call for logic that doesn't need one.

Evidence

- `docs/baseline-metrics.md` — baseline weakness that motivated the original rule-writing effort
- `docs/before-after-risk-scoring-conversion.md` — measured latency (0.0011 ms/call, real measurement) and cost ($0) for the deterministic version, plus qualitative predictability/maintainability/audit-clarity comparison

- `skills/risk_scoring.py`'s self-test — 9/9 rule-branch cases pass, directly runnable in CI without any LLM API dependency

Consequences

- Faster, free, and fully deterministic — same findings always produce the same recommendation
- Directly unit-testable in CI (`.github/workflows/policy-checks.yml` can run `skills/risk_scoring.py` as a pure Python check, no API mocking needed)
- Structurally cannot reintroduce the baseline's Risk Flagging inconsistency, since there's no LLM reasoning step left to vary

Positive:

- Loses flexibility for a genuinely novel finding pattern that doesn't cleanly fit the existing severity categories — the code will apply the rule mechanically rather than exercising judgment on an edge case
- If the underlying rules ever need to become more nuanced (e.g., weighting *combinations* of findings differently, not just the single highest severity), the deterministic function will need active maintenance rather than the agent "figuring it out"

Negative / Trade-offs:

Open Risks

- This conversion assumes the reviewer agent's own `findings` (severity assignment per issue) remain agent-judgment, which is where the actual risk of misjudgment now concentrates. If the reviewer over- or under-rates a finding's severity, the deterministic mapping will faithfully carry that error through — this ADR does not address finding-level severity accuracy, only the mapping step downstream of it
- Not yet validated against real reviewer-agent output at scale (the orchestrator/pipeline isn't built yet — see `docs/gap-inventory.md`'s "Known Remaining Gap"). This ADR's evidence is based on the rule table's own logical completeness and a directly measured function benchmark, not a live pipeline comparison.

ADR-002: Five-Dimension Quality Rubric Design

Status: *Accepted*

Date: *2026-08-02*

Context

The project needed a way to score any PR review (human or agent) against consistent, comparable criteria — both for setting the Module 1 baseline and for later evaluating the agentic pipeline against it. Without a shared rubric, "the agent is better than a human" would be an unfalsifiable claim.

Decision

Adopt a 5-dimension rubric, each scored 0-2: Bug Detection, Risk Flagging, False Positive Control, Grounding, and Release Note Quality. Pass threshold set at >=8/10 total, with a hard requirement of 2/2 on Bug Detection whenever a seeded bug is present in the input.

Rejected Alternatives

Alternative 1: A single overall 1-5 quality score. Rejected because a single number can't distinguish *why* a review scored poorly — a review that misses a bug and a review that's vague but technically correct would both just show "3/5," with no diagnostic value for improving the pipeline.

Alternative 2: Binary pass/fail per PR, no partial credit. Rejected as too coarse for a small ground-truth set — with only a handful of PRs in the baseline sample, a binary scheme would produce too little signal to compare baseline vs. agent meaningfully. The 0-2 scale per dimension gives finer-grained comparison on a small sample.

Alternative 3: Score only Bug Detection, since that's the core task. Rejected once the baseline run was actually completed — evidence: `docs/baseline-run-notes.md` showed human reviewers caught 3/3 seeded bugs (perfect Bug Detection) while still scoring only 1/2 on Risk Flagging across every case. A rubric that only measured Bug Detection would have completely missed this real, measured weakness, which later became the direct motivation for the risk-scoring deterministic conversion (ADR-001).

Evidence

`docs/baseline-run-notes.md` — the rubric's multi-dimension design is what allowed the Risk Flagging weakness to be measured and later fixed; a coarser rubric would not have surfaced it.

Consequences

Positive: dimension-level scoring gave a specific, actionable target (Risk Flagging) rather than a vague "do better" signal, and directly enabled ADR-001's deterministic conversion to be evidence-based rather than speculative.

Negative: self-scoring by a single human reviewer (no second scorer) means the baseline scores have some subjectivity; a larger project would want inter-rater reliability checks.

Open Risks

The rubric's thresholds (8/10 pass, 2/2 Bug Detection requirement) were set by judgment, not derived from a larger calibration dataset — they may need adjustment once more PRs are scored.

ADR-003: Memory Layout — Separate Review History, Calibration Log, and Session Context

Status: *Accepted*

Date: *2026-08-02*

Context

The pipeline needed persistent memory so the reviewer agent doesn't re-derive judgment from scratch on every run, without conflating three different kinds of state: what's fixed per-agent (prompt), what's specific to one run (context), and what should persist and inform future runs (memory).

Decision

- **Review history** (`memory/store/review-history.jsonl`) — append-only, long-term, one line per past PR review outcome
- **Calibration log** (`memory/store/calibration-log.jsonl`) — append-only, long-term, records prompt/skill/tool-grant changes made because of evidence
- **Session context** — short-term, exists only for the duration of one pipeline run (the current diff, in-progress findings), never persisted

Split memory into three explicit types, each with a different persistence lifetime and storage format:

Rejected Alternatives

Alternative 1: A single flat memory file for everything. Rejected because review outcomes and calibration decisions have different write patterns and different audiences (review history is written by every run; calibration log entries are rare and require human promotion — see ADR-006). Mixing them would make it harder to enforce the write-access distinction MCP's `TOOL_GRANTS` actually needs (see ADR-004).

Alternative 2: A real database (SQLite) instead of JSON Lines files. Rejected as unnecessary complexity for this project's scale — evidence: the entire review-history dataset produced during this project is a handful of records (see `memory/store/review-history.jsonl`), well within what a flat, human-readable, git-diffable file handles fine. JSON Lines also makes every record individually appendable and readable without a query layer, which matters for the audit-trail requirement.

Alternative 3: Store full PR diffs in memory alongside the outcome. Rejected for both size and safety reasons — diffs can be large, and persisting full source code in a long-lived memory store increases the risk of accidentally retaining something sensitive. `memory/architecture-notes.md` explicitly documents this as "What is explicitly NOT stored," storing only summaries with a reference back to the source diff file instead.

Evidence

`memory/architecture-notes.md`'s explicit table of what's stored by whom, and the working `mcp/storage_server.py` schema validation, which rejects any record missing required fields — this only works cleanly because review history and calibration data have distinct, well-defined schemas.

Consequences

Positive: clean separation makes the MCP tool-grant boundaries (ADR-004) straightforward to reason about — different memory types can have different read/write rules without special-casing.

Negative: three files instead of one adds a small amount of bookkeeping overhead (e.g., needing to know which file a given fact belongs in).

Open Risks

At larger scale (many more PRs), flat JSON Lines files may need to move to an indexed store for query performance — not a concern at this project's current data volume, but worth flagging for a real deployment.

ADR-004: Two Separate MCP Servers, Local Stdio Transport

Status: *Accepted*

Date: *2026-08-04*

Context

The pipeline needed MCP-backed persistent storage and vector retrieval, per Workstream 3's requirements. A decision was needed on how many servers to run, and how agents would connect to them.

Decision

Run two separate local MCP servers over stdio JSON-RPC: `pr-review-storage` (memory read/write) and `pr-review-retrieval` (TF-IDF search). Each server independently enforces its own `TOOL_GRANTS` role-based allow-list on every call, rather than trusting a single shared gatekeeper.

Rejected Alternatives

Alternative 1: One combined MCP server for both storage and retrieval. Rejected because storage and retrieval have meaningfully different data classifications (storage is "internal" — the pipeline's own judgment data; retrieval is "public" — sourced from public PR diffs, per `docs/governance-policy.md`'s classification table). Combining them into one server would make it harder to reason about — and easier to accidentally misconfigure — which classification applies to which tool.

Alternative 2: Use the official `mcp` Python SDK instead of a hand-rolled JSON-RPC implementation. This was the original intent, but rejected in practice — evidence: the project's container has no network access to install packages from PyPI at the point these servers needed to run, documented directly in `mcp/storage_server.py`'s "PROTOCOL NOTE." The hand-rolled implementation matches the SDK's exact request/response shape (`initialize`/`tools/list`/`tools/call` over newline-delimited JSON-RPC), so swapping to the real SDK later requires no change to tool names, schemas, or permission boundaries — only the transport layer.

Alternative 3: Trust a single orchestrator-level permission check instead of having each MCP server independently enforce `TOOL_GRANTS`. Rejected on defense-in-depth grounds: if the orchestrator's own permission check were ever bypassed or misconfigured, a single point of enforcement would mean no boundary at all. Each server checking its own `_caller_role` argument means the boundary holds even if the calling code is wrong — confirmed directly by the tool-evolution drill (`docs/tool-evolution-drill.md`), where a real permission regression was caught precisely because the server itself, not just documentation, enforces the check.

Evidence

`docs/tool-evolution-drill.md` — the drill only worked as a meaningful test because permission enforcement lives in the server code itself, not in a document or a single trusted caller.

Consequences

Positive: independently testable, independently enforced boundaries; `tests/test_policy.py` can verify each server's `TOOL_GRANTS` separately.

Negative: two servers to run and keep in sync with the governance policy doc, rather than one — mitigated by the CI policy test catching drift automatically (Section 8/9 of the architecture write-up).

Open Risks

The hand-rolled JSON-RPC implementation has not been tested against a real MCP client beyond this project's own `orchestrator.py` and manual subprocess calls; swapping to the official SDK in a networked environment would be the

natural next validation step.

ADR-005: Three-Agent Split with Mandatory Reviewer Routing

Status: *Accepted*

Date: *2026-08-02*

Context

The PRD identified five possible roles (planner, implementer, reviewer, tester, release-manager). A decision was needed on how many subagents to actually build for the capstone, how work would route between them, and what would prevent a PR from bypassing review.

Decision

Build three subagents — `planner`, `reviewer`, `release-manager` — with a fixed routing rule enforced by the orchestrator: the planner always runs first, must always hand off to the reviewer (this cannot be skipped), and the release-manager only runs after the reviewer has produced findings.

Rejected Alternatives

Alternative 1: Build all five PRD-listed roles (add implementer and tester). Rejected for this capstone's scope — the three built roles already demonstrate a real orchestrator-and-subagents story (routing, scoped tools, hand-off rules) without the added surface area of two more roles whose value-add (writing fixes, running tests) is less central to the core "catch problems, draft notes" workflow this project targets.

Alternative 2: Let the planner decide dynamically whether to invoke the reviewer, based on its own judgment of PR risk. Rejected as a governance risk — evidence: this is exactly the kind of decision that, if left to agent judgment, could let a low-effort or miscalibrated planner skip review on a PR that actually needed it. `agents/planner.md` makes this explicit in its "Known Failure Modes" section ("Skipping review... must be structurally prevented, not just discouraged"), and `docs/orchestration-diagram.md`'s routing rules make this an orchestrator-level rule, not a planner-level choice.

Alternative 3: Let release-manager run independently of the reviewer, drafting a note directly from the diff. Rejected because a release note written without knowing the reviewer's findings could describe a change as safe when the reviewer flagged it as risky. `agents/release-manager.md`'s behavior rules explicitly require reading the reviewer's `overall_recommendation` and adjusting tone if escalated — this is only possible if release-manager strictly runs after reviewer.

Evidence

`docs/orchestration-diagram.md`'s routing rules section; `agents/planner.md`'s explicit "must invoke reviewer" behavior rule, cross-referenced against the orchestrator's actual control flow in `orchestrator.py`, which always calls `run_reviewer()` before `run_release_manager()`.

Consequences

Positive: the mandatory-review rule is a hard structural boundary, not a soft convention — the same design pattern as the MCP permission boundaries (ADR-004), applied at the routing level.

Negative: less flexible than a fully dynamic routing scheme — every PR goes through the same fixed sequence, even ones a more adaptive system might handle differently (e.g., trivially small documentation-only changes still go through the full reviewer step).

Open Risks

The fixed routing sequence hasn't been stress-tested against PRs that might genuinely benefit from a different order (e.g., a PR so large it should be split before review) — this is a known simplification for the capstone's scope, not a claim that fixed routing is optimal for every real-world case.

ADR-006: Least-Privilege Defaults with Human-Gated Calibration Promotion

Status: *Accepted*

Date: *2026-08-05*

Context

The project needed a governance model that is actually enforced, not just documented — per the capstone rubric's explicit requirement that "a policy document alone is not enough." A decision was needed on default access levels per role, and specifically on how the calibration log (which records changes made because of eval evidence) gets written to, since that log is itself a record of the system modifying its own behavior.

Decision

Every role defaults to zero access; grants are added only with specific justification (see `docs/governance-policy.md`'s "Justification-to-Widen Process"). Critically, no agent tool writes directly to `calibration-log.jsonl` — agents may only call `put_calibration_proposal`, which writes to a separate proposals file. A human must promote an accepted proposal into the real calibration log.

Rejected Alternatives

Alternative 1: Grant broad default access and restrict only where a problem is found. Rejected as the opposite of least-privilege — this would mean every new tool or capability is accessible by default until someone notices it shouldn't be, rather than inaccessible by default until justified. The capstone's own governance requirements call for least-privilege defaults explicitly.

Alternative 2: Let any agent write directly to the calibration log when it determines a change is justified. Rejected specifically because the calibration log is a record of the system changing its own future behavior (routing rules, prompts, tool grants) — allowing an agent to both decide a change is warranted *and* commit that change unilaterally removes the human check on self-modification entirely. The `put_calibration_proposal` / human-promotion split keeps a human in the loop for exactly this class of decision, matching the escalation principle already applied elsewhere (e.g., `skills/risk-scoring.md`'s rule that critical findings escalate to a human rather than self-resolving).

Alternative 3: Enforce least-privilege only via documentation and code review, without a CI-testable check. Rejected once the tool-evolution drill was actually run — evidence: `docs/tool-evolution-drill.md` shows a real, specific permission regression that a documentation-only policy would not have caught until it reached production. `tests/test_policy.py` running in CI (`.github/workflows/policy-checks.yml`) catches this class of drift before merge, which a purely human-review-based process cannot guarantee.

Evidence

`docs/tool-evolution-drill.md` — direct evidence that CI-enforced policy checks catch what documentation-only governance would have missed; `memory/store/calibration-proposals.jsonl`'s real logged proposal (a retrieval-threshold suggestion from `docs/retrieval-quality-report.md`'s findings), showing the propose-then-promote flow working as designed.

Consequences

Positive: self-modification of the system's own behavior always passes through a human checkpoint; the CI test suite catches accidental drift, not just deliberate attacks.

Negative: every new legitimate capability requires an explicit grant-widening step rather than working by default — a minor friction cost traded for the safety guarantee.

Open Risks

The human-promotion step for calibration proposals is not yet exercised at any real scale (only one proposal has been logged so far) — its practicality under a higher volume of proposals is untested.